

Análisis de datos en HDFS con Hive

Adrián Robles Arques

Introducción

En el presente trabajo vamos a aprender a gestionar archivos de datos tabulares empleando el sistema de archivos distribuidos HDFS (Hadoop Distributed File System) y a crear bases de datos en este sistema, así como realizar consultas, empleando la herramienta Hive, también del ecosistema Hadoop.

Consideraciones preliminares

Esta práctica se va a realizar empleando una máquina virtual del sistema Ubuntu, la cual nos aporta IEBS. En la memoria de dicha máquina ya se encuentran los datos a emplear, por lo que no necesitaremos descargar ningún fichero. Además, las herramientas y paquetes necesarios ya se encuentran instalados.

En este caso, disponemos de los datos en formato parquet de diferentes aeropuertos por todo el mundo. Los datos se estructuran en una tabla siguiendo este esquema:

- Airport ID: Número identificador único de cada aeropuerto.
- Name: Nombre del aeropuerto, que puede o no contener el nombre de la ciudad.
- City: Ciudad donde se ubica la terminal, o principal ciudad a la que sirve.
- Country: País donde se ubica el aeropuerto.
- IATA: Código de tres letras, nulo si es desconocido o no tiene código asignado.
- ICAO: Código de cuatro letras, nulo si es desconocido o no tiene código asignado.
- Offset: Horas de desviación respecto a UTC, contiene decimales.
- DST: Código de una letra que indica la región de DST que aplica el aeropuerto.
- Timezone: Zona horaria indicada por la región en la que se encuentra.
- Type: Tipo de terminal, en este caso solo se incluyen aeropuertos.
- Source: Fuente de los datos.

Por tanto la base de datos a crear se hará de acuerdo a estas columnas y la tipología de datos que podemos ubicar en cada una de ellas.

Procedimiento

A continuación se exponen los pasos seguidos para cargar los datos en HDFS, crear la tabla en Hive y generar el puntero que conecta con los datos alojados en el sistema distribuido. Una vez arrancada la máquina virtual, comenzamos a trabajar con la terminal de Linux.

Creación de la base de datos en HDFS

Para comenzar el procedimiento, iniciamos HDFS en la terminal. Para ello, cambiamos al directorio de instalación de Hadoop y ejecutamos el comando de inicio del sistema distribuido de datos “start-dfs.sh”.

Este comando inicia los Name Nodes en el localhost, porque en este caso lo usaremos como servidor para la base de datos, así como los Data Nodes. A continuación, emplearemos el comando de HDFS “dfs -mkdir” para crear el directorio donde alojaremos el contenido de la base de datos, asignando el nombre y la ruta indicada:

```
bigdata@Big-Data:~$ cd /home/bigdata/hadoop-3.3.6/
bigdata@Big-Data:~/hadoop-3.3.6$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Big-Data]
bigdata@Big-Data:~/hadoop-3.3.6$ bin/hdfs dfs -mkdir -p /data/airports/
bigdata@Big-Data:~/hadoop-3.3.6$
```

A continuación vamos a cargar los datos en la carpeta creada en HDFS desde el archivo en formato parquet alojado en la máquina virtual. Previamente vamos a localizar el archivo en Ubuntu, en mi caso he cambiado el nombre para que pueda referenciarse fácilmente desde la terminal y no carguemos otros ficheros, en este caso lo he renombrado como ‘datos_aeropuertos.parquet’.

Una vez localizada la ruta y el documento, procedemos a cargarlo en HDFS empleando el comando ‘dfs -copyFromLocal’. A continuación indicamos la ruta del fichero a copiar, seguido de la ruta de destino en el sistema de archivos distribuidos:

```
bigdata@Big-Data:~/hadoop-3.3.6$ bin/hdfs dfs -copyFromLocal /home/bigdata/Descargas/aeropuertos/datos_aeropuertos.parquet /data/airports/
bigdata@Big-Data:~/hadoop-3.3.6$
```

Para comprobar que los datos se han cargado correctamente, vamos a listar en la terminal los datos que contiene la carpeta de destino, para ello emplearemos el comando ‘dfs -ls’ seguido del nombre de la carpeta a mostrar:

```
bigdata@Big-Data:~/hadoop-3.3.6$ bin/hdfs dfs -ls /data/airports/
Found 2 items
drwxr-xr-x  - bigdata supergroup          0 2025-05-14 09:56 /data/airports/aeropuertos
-rw-r--r--  1 bigdata supergroup    435853 2025-05-14 11:40 /data/airports/datos_aeropuertos.parquet
```

En este caso vemos que el fichero ‘datos_aeropuertos.parquet’ se encuentra en la carpeta deseada, por lo que hemos completado la carga de los datos*. Podemos comprobarlo también en el navegador, acudiendo al localhost.

***Nota:** Dentro de ‘/data/airports/’ aparece otra carpeta porque previamente había copiado la carpeta completa, dado que en su interior se encuentra otro fichero. Finalmente este fichero no fue necesario y cargué de nuevo los datos en la carpeta, pero únicamente el archivo ‘datos_aeropuertos.parquet’.

Show entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	drwxr-xr-x	bigdata	supergroup	0 B	May 14 09:56	0	0 B	aeropuertos	
☐	-rw-r--r--	bigdata	supergroup	425.64 KB	May 14 11:40	1	128 MB	datos_aeropuertos.parquet	

Showing 1 to 2 of 2 entries

Efectivamente, accediendo al sistema HDFS desde el localhost también podemos ver que los datos de los aeropuertos se han cargado completamente. Por lo tanto, el siguiente paso será cargarlos en una tabla en Hive.

Una vez cargados los datos en el sistema de archivos distribuidos, vamos a crear una tabla externa en Hive. En este caso, al tratarse de una tabla externa, los datos serán gestionados por HDFS, mientras que Hive controlará la estructura y metadatos asociados a los mismos. De esta forma, cada vez que hagamos una consulta Hive sabrá ubicar los datos en HDFS y procesarlos, al conocer cómo están almacenados.

```

[SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Connecting to jdbc:hive2://
Hive Session ID = 4276242f-b0c7-4c42-a7d3-d3f27d94338c
25/05/12 19:22:50 [main]: WARN session.SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is set to instance of HiveAuthorizerFactory.
25/05/12 19:22:50 [main]: WARN metastore.ObjectStore: datanucleus.autoStartMechanismMode is set to unsupported value null . Setting it to value: ignored
25/05/12 19:22:50 [main]: WARN util.DriverDataSource: Registered driver with driverClassName=org.apache.derby.jdbc.EmbeddedDriver was not found, trying direct instantiation.
25/05/12 19:22:51 [main]: WARN util.DriverDataSource: Registered driver with driverClassName=org.apache.derby.jdbc.EmbeddedDriver was not found, trying direct instantiation.
25/05/12 19:22:51 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:51 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:51 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:51 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:51 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:51 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:52 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:52 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:52 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:52 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:52 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:52 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
25/05/12 19:22:52 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid. Ignored
Connected to: Apache Hive (version 3.1.3)
Driver: Hive JDBC (version 3.1.3)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 3.1.3 by Apache Hive
0: jdbc:hive2://>

```

A continuación, será necesario crear la tabla externa en Hive. Hive utiliza un lenguaje muy similar a SQL, para crear la tabla empleamos el comando `CREATE EXTERNAL TABLE`, indicando que es una tabla externa, seguido del nombre de la tabla.

A continuación deberemos instanciar todas las columnas, con nombre y tipo de dato, tal como se indicaba en las consideraciones preliminares. Por último, indicaremos el tipo de archivo que contiene los datos y su ubicación. A continuación se muestra el proceso con el código completo:

```
0: jdbc:hive2://> CREATE EXTERNAL TABLE aeropuertos(
. . . . . > airport_id int,
. . . . . > name string,
. . . . . > city string,
. . . . . > country string,
. . . . . > IATA string,
. . . . . > ICAO string,
. . . . . > Latitude float,
. . . . . > longitude float,
. . . . . > altitude bigint,
. . . . . > UTC_offset float,
. . . . . > DST string,
. . . . . > Timezone string,
. . . . . > type string,
. . . . . > source string)
. . . . . > STORED AS parquet
. . . . . > LOCATION '/data/airports/'
. . . . . > ;
OK
No rows affected (0,28 seconds)
0: jdbc:hive2://>
```

Vamos a mostrar las tablas gestionadas por Hive para comprobar que la tabla se ha creado adecuadamente, usando para ello el comando 'SHOW TABLES':

```
0: jdbc:hive2://> SHOW TABLES;
OK
+-----+
| tab_name |
+-----+
| aeropuertos |
+-----+
1 row selected (0,355 seconds)
0: jdbc:hive2://>
```

Con esto podemos comprobar que la tabla se ha creado correctamente, con el formato de tabla externa y con las columnas indicadas, cargando los datos desde la ubicación indicada. De no ser así, habría aparecido un error durante la creación de la tabla y no se mostraría.

Consultas a la base de datos

A continuación voy a realizar una serie de consultas a la base de datos para responder a las preguntas planteadas durante el sprint. Para ello se emplea el lenguaje HQL, muy similar al procedimiento en una base de datos estructurada en SQL.

Pregunta 1: ¿Qué aeropuerto se encuentra a mayor altitud?

Para responder a esta pregunta, vamos a extraer los datos de identificación del aeropuerto, ordenarlos de forma descendente en base a la altitud a la que se ubican, medida en pies, y mostrar los diez primeros resultados para que sea fácilmente visible en pantalla.

Consulta:

```
0: jdbc:hive2://> SELECT airport_id, name, city, country, altitude FROM aeropuertos
. . . . . > ORDER BY altitude DESC
. . . . . > LIMIT 10;
```

Resultado:

```
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 1719232 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
```

airport_id	name	city	country	altitude
NULL	Daocheng Yading Airport	Daocheng	China	14472
NULL	Qamdo Bangda Airport	Bangda	China	14219
NULL	Kangding Airport	Kangding	China	14042
NULL	Ngari Gunsai Airport	Shiquanhe	China	14022
NULL	El Alto International Airport	La Paz	Bolivia	13355
NULL	Capitan Nicolas Rojas Airport	Potosi	Bolivia	12913
NULL	Yushu Batang Airport	Yushu	China	12816
NULL	Copacabana Airport	Copacabana	Bolivia	12591
NULL	Inca Manco Capac International Airport	Juliaca	Peru	12552
NULL	Golog Maqin Airport	Golog	China	12426

```
10 rows selected (1,364 seconds)
```

El primero de estos resultados será, por tanto, el aeropuerto que se encuentra a mayor altitud, siendo este el Daocheng Yading Airport, en Daocheng, China.

Pregunta 2: ¿Cuántos aeropuertos hay en España?

Para responder a esta consulta vamos a realizar un conteo sobre la columna países, de modo que sumemos 1 cada vez que en la misma aparezca 'Spain'.

Consulta:

```
0: jdbc:hive2://> SELECT COUNT(country)
. . . . . > FROM aeropuertos
. . . . . > WHERE country = 'Spain';
```

Respuesta:

```

MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 1811168 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
+-----+
|  _c0  |
+-----+
|  64   |
+-----+

```

Visto el resultado, podemos decir que de acuerdo a nuestra base de datos, España tiene un total de 64 aeropuertos.

Pregunta 3: ¿Qué países tienen aeropuertos cuyo horario de verano sea el de Europa?

Para esta consulta vamos a hacer un proceso de selección, en el cual se seleccionarán únicamente los países para los que al menos un aeropuerto presente en la columna de DST, o Daylight Saving Time, la indicación de 'E', que significa que se acogen al horario europeo. Para que únicamente se cuente una vez a cada país, aunque presente más de un aeropuerto donde esto sea cierto, empleamos la función DISTINCT() tal como se muestra en la consulta:

Consulta:

```

0: jdbc:hive2://> SELECT DISTINCT(country)
. . . . . > FROM aeropuertos
. . . . . > WHERE dst = 'E';

```

Respuesta:

country
Albania
Armenia
Austria
Azerbaijan
Belarus
Belgium
Bosnia and Herzegovina
Bulgaria
Croatia
Cyprus
Czech Republic
Denmark
Egypt
Estonia
Faroe Islands
Finland
France
Germany
Greece
Greenland
Guernsey
Guinea
Hungary
Iceland
Iran
Ireland
Isle of Man
Israel
Italy
Jersey
Jordan
Kyrgyzstan
Latvia
Lebanon
Lithuania
Luxembourg
Macedonia
Malta
Moldova
Montenegro
Morocco
Netherlands
Norway
Poland
Portugal
Reunion
Romania
Russia
Serbia
Slovakia
Slovenia
Spain
Sweden
Switzerland
Syria
Tunisia
Turkey
Ukraine
United Kingdom
United States
Uzbekistan

Como aparecen varios territorios donde, a priori, no esperaba encontrar el horario de verano de Europa, dado que en principio se encuentran principalmente en otras zonas geográficas que tienen su propio horario DST, he realizado una consulta secundaria para que muestre el aeropuerto en cuestión. En este caso, he realizado la consulta sobre Estados Unidos, el país que más me choca de la lista, por su gran distancia geográfica respecto de Europa.

Para esta consulta secundaria voy a mostrar todos los aeropuertos cuyo país sea EEUU, y que tengan horario de verano europeo.

Consulta:

```
0: jdbc:hive2://> SELECT DISTINCT(country), airport_id, city, country, name, dst
. . . . . > FROM aeropuertos
. . . . . > WHERE dst = 'E' AND country = 'United States';
```

Respuesta:

```
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 389368 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
+-----+-----+-----+-----+-----+-----+
| country | airport_id | city | country | name | dst |
+-----+-----+-----+-----+-----+-----+
| United States | NULL | El Monte | United States | San Gabriel Valley Airport | E |
+-----+-----+-----+-----+-----+-----+
1 row selected (3,599 seconds)
0: jdbc:hive2://>
```

Al parecer hay un Aeropuerto, ubicado en El Monte, Los Ángeles, cuya zona DST está establecida como Europea (E), en lugar de EEUU/Canadá (A). La diferencia es la fecha de inicio del horario de verano, ya que aunque es una medida que se aplica por igual en Norteamérica y Europa, la fecha de inicio presenta una variación de dos semanas. Al parecer, este aeropuerto en concreto inicia el horario de verano concordando con la fecha europea, según esta base de datos.

Conclusiones

Aunque en esta práctica, a modo de prueba, hemos desplegado HDFS en la propia máquina virtual, en producción este sistema podría desplegarse en varios servidores con distintas ubicaciones geográficas, y realizando copias de los archivos en diferentes nodos para aumentar la seguridad.

Aún así, desde el gestor de tablas, como en este caso Hive, veríamos una única tabla de datos (O las que hayamos creado), y nos permitiría lanzar consultas empleando las funciones de MapReduce a centros de datos distribuidos geográficamente.

Por tanto, podemos comprobar cómo un sistema de datos distribuido como HDFS es un pilar fundamental para el crecimiento horizontal de proyectos y el establecimiento de un sistema globalizado con mayor seguridad, al no distribuir la información guardada, y mayor velocidad de consulta, al operar de forma prioritaria con los nodos más próximos.